nextBooq Frontend Architecture & Folder Structure

This document outlines the complete Next.js (App Router) folder structure. It is designed to cleanly separate the three main sides of the platform while sharing reusable UI components.

1. Root & Configuration

The top level of your repository containing configuration files.

```
/
├── .env.local                  # Environment variables (API URL, Razorpay Keys)
├── middleware.ts               # CRITICAL: Intercepts requests. Rewrites `coffeec
├── tailwind.config.ts          # Tailwind CSS configuration and theme variables.
├── components.json             # shadcn/ui configuration file.
├── package.json                # Dependencies (Next.js, React Query, Zustand, Axi
└── tsconfig.json               # TypeScript configuration.
```

2. App Router (`src/app`)

This is the core routing logic. We use Route Groups `(...)` to separate layouts without affecting the URL, and Dynamic Segments `[...]` to capture the cafe slug from the subdomain.

```
src/app/
├── layout.tsx                  # Root HTML/Body layout. Wraps the entire app in <
├── globals.css                 # Global CSS, Tailwind imports, CSS variables for
│
├── (control-plane)/            # SIDE 1: nextBooq's Main Platform (www.nextbooq.c
│   ├── layout.tsx              # Layout for the marketing site (Header, Footer).
│   ├── page.tsx                # Marketing Landing Page.
│   ├── pricing/page.tsx        # Pricing page.
│   ├── auth/
│   │   ├── login/page.tsx      # Platform login (Users & Cafe Owners).
│   │   └── register/page.tsx   # Platform signup.
│   └── onboarding/
│       ├── layout.tsx          # Specific layout for the onboarding wizard.
│       └── page.tsx            # Step-by-step wizard calling POST /api/v1/cafes.
│
└── [tenant]/                   # Multi-Tenant Routes (*.nextbooq.com)
    ├── layout.tsx              # Fetches Cafe details via `slug`. Wraps children
    │
    ├── (public)/               # SIDE 2: Customer Booking Page (coffeecove.nextbo
    │   ├── page.tsx            # The main reservation wizard (Select Party Size &
    │   ├── checkout/page.tsx   # The Razorpay payment confirmation page.
    │   └── success/page.tsx    # Reservation confirmed (QR Code, details).
    │
    └── (dashboard)/            # SIDE 3: Cafe Admin/Host Stand (coffeecove.nextbo
        ├── layout.tsx          # Dashboard shell (Sidebar navigation, Header with
        ├── page.tsx            # Main Host Stand view (Manifest + Floorplan).
        ├── reservations/page.tsx # Detailed list/history of reservations.
        ├── tables/page.tsx     # Inventory management (Create/Edit Tables, can_me
        └── settings/page.tsx   # Cafe settings (Rules, Turn Times, Timezone).
```

3. UI Components (`src/components`)

This folder is strictly for React components. It is divided by functional domain.

```
src/components/
├── ui/                         # "Dumb" shared components (Generated by shadcn/ui
│   ├── button.tsx
│   ├── input.tsx
│   ├── dialog.tsx              # Modals
│   ├── toast.tsx               # Notifications
│   └── calendar.tsx
│
├── booking/                    # Customer Booking Flow Components
│   ├── time-slot-picker.tsx    # Renders available times. Hooks into `useAvailabi
│   ├── party-size-select.tsx   # Number input restricted by cafe rules.
│   ├── auth-overlay.tsx        # Prompt to login/register before holding a table.
│   └── razorpay-checkout.tsx   # Handles the Razorpay JS SDK integration.
│
├── dashboard/                  # Host Stand & Admin Components
│   ├── manifest-table.tsx      # The daily list of bookings.
│   ├── floorplan-canvas.tsx    # Visual map of tables. Turns red/green based on s
│   ├── status-badge.tsx        # Small UI pill for "SEATED", "CONFIRMED", etc.
│   ├── table-editor-modal.tsx # Form to create/edit a physical table.
│   └── reservation-drawer.tsx # Slide-out panel to update reservation status.
│
├── control-plane/              # Marketing & Onboarding Components
│   ├── hero-section.tsx
│   ├── pricing-cards.tsx
│   └── onboarding-wizard.tsx   # Multi-step form managing state via Zustand.
│
└── providers/                  # React Context Providers
    ├── query-provider.tsx      # Sets up TanStack React Query Client.
    ├── auth-provider.tsx       # Exposes current user and permissions globally.
    └── tenant-provider.tsx     # Reads cafe branding from props and sets CSS vari
```

4. API & Data Fetching (`src/hooks`)

This folder contains all TanStack React Query custom hooks. Every Express endpoint from your API spec has a corresponding hook here.

```
src/hooks/
├── api/                        # API Client configuration
│   └── axios-instance.ts       # Axios instance pre-configured with base URL and
│
├── auth/                       # Maps to Auth Module
│   ├── use-login.ts            # useMutation for POST /auth/login
│   ├── use-register.ts         # useMutation for POST /auth/register
│   └── use-me.ts               # useQuery to fetch current user profile from cook
│
├── public/                     # Maps to Public Customer Module
│   ├── use-cafe-details.ts     # useQuery for GET /cafes/slug/:slug
│   ├── use-availability.ts     # useQuery for GET /cafes/:cafeId/available-slots
│   └── use-book-table.ts       # useMutation for POST /cafes/:cafeId/reservations
│
├── host/                       # Maps to Host Stand Module
│   ├── use-manifest.ts         # useQuery for GET /cafes/:cafeId/reservations?dat
```

```
│   └── use-update-status.ts   # useMutation for PUT .../reservations/:resId/stat
│
└── admin/                     # Maps to Admin Inventory Module
    ├── use-create-cafe.ts     # useMutation for POST /cafes
    ├── use-tables.ts          # useQuery for GET /cafes/:cafeId/tables
    ├── use-create-table.ts    # useMutation for POST /cafes/:cafeId/tables
    └── use-edit-table.ts      # useMutation for PUT /cafes/:cafeId/tables/:table
```

5. Local State Management (`src/store`)

This folder contains Zustand stores for handling transient, UI-specific state that doesn't belong in the database or URL.

```
src/store/
├── use-booking-store.ts       # Stores the customer's progress during checkout.
│                              # State: { date, time, partySize, step: 1|2|3 }
│                              # Actions: setDate(), reset()
│
├── use-dashboard-store.ts     # Stores Host Stand UI state.
│                              # State: { selectedReservationId, isDrawerOpen }
│
└── use-onboarding-store.ts    # Stores the cafe owner's progress during signup.
                               # State: { cafeName, slug, selectedPlan }
```

6. Utilities (`src/lib`)

Helper functions and type definitions.

```
src/lib/
├── utils.ts                   # Standard Tailwind merge utilities (clsx, twMerge
├── timezone.ts                # Wrappers around `date-fns-tz` to ensure strict U
├── razorpay.ts                # Helper to load Razorpay script and verify signat
└── types/                     # TypeScript Interfaces
    ├── api-responses.d.ts     # Interfaces matching your Express JSON responses.
    ├── models.d.ts            # Interfaces for Cafe, User, Table, Reservation.
    └── auth.d.ts              # Interfaces for JWT payloads and Role Enums.
```